

Image Manipulation Localization Using Dual-Shallow Feature Pyramid Fusion and Boundary Contextual Incoherence Enhancement

Yan Xiang¹, Xiaochen Yuan¹, Senior Member, IEEE, Kaiqi Zhao, Tong Liu², Graduate Student Member, IEEE, Zhiyao Xie¹, Guoheng Huang¹, and Jianqing Li¹, Senior Member, IEEE

I. INTRODUCTION

Abstract—This paper proposes a novel end-to-end network for Image Manipulation Localization (IML) comprising three modules: feature fusion, encoder, and decoder. To address the limitations of current DNN-based IML algorithms in accessing global features and segmenting tampered edges, we propose a Dual-shallow Feature Pyramid Fusion (DFPF) module. The DFPF module integrates semantic and texture features through a bidirectional pathway, forming RGB Feature Pyramids (RGBFP) and Local Textual Feature Pyramids (LTFP) using dual Hybrid ResNet50s in a 'Siamese' configuration. These feature pyramids are merged via multi-scale fusion to enhance global pyramid features for decoding. The LTFP branch includes a Pre-processing Block, Parallel Multi-Scale Convolution (PMSC), or Channel Split High-frequency Convolution (CSHC) to capture local textual features and subtle manipulation traces. The Encoder employs Transformer layers for robust global representations. At the same time, the Decoder uses Cascaded Boundary Context Inconsistent Enhancement (BCIE) Blocks to reconstruct a coarse-to-fine binary mask, enhancing texture inconsistencies at manipulated region boundaries. Additionally, we introduce an automated method for generating a large-scale forgery dataset via Photoshop Scripting, reducing labor costs. Our model effectively locates tampered regions of various shapes and sizes, improving boundary anomaly detection. Extensive experimental results demonstrate that our method significantly outperforms existing state-of-the-art models.

Index Terms—Image manipulation localization (IML), dual-shallow feature pyramid fusion (DFPF), boundary contextual incoherence enhancement (BCIE), parallel multi-scale convolution (PMSC), channel split high-frequency convolution (CSHC).

Received 24 May 2024; revised 1 August 2024; accepted 7 September 2024. Date of publication 29 November 2024; date of current version 24 July 2025. This work was supported in part by the Science and Technology Development Fund of Macau SAR under Grant 0045/2022/A and in part by the Macao Polytechnic University under Grant RP/FCA-12/2022. (Corresponding authors: Xiaochen Yuan; Jianqing Li.)

Yan Xiang and Jianqing Li are with the School of Computer Science and Engineering, Faculty of Innovation Engineering, Macau University of Science and Technology, Macau SAR 999078, China (e-mail: 2009853gia30006@student.must.edu.mo; jqli@must.edu.mo).

Xiaochen Yuan, Tong Liu, and Zhiyao Xie are with the Faculty of Applied Sciences, Macao Polytechnic University, Macau SAR 999078, China (e-mail: xcyuan@mpu.edu.mo; p2209360@mpu.edu.mo; p2215884@mpu.edu.mo).

Kaiqi Zhao is with the School of Cyber Security, Shandong University of Political Science and Law, Jinan 250013, China (e-mail: kqzhao@sdupsl.edu.cn).

Guoheng Huang is with the School of Computer Science and Technology, Guangdong University of Technology, Guangzhou 510006, China (e-mail: kevinwong@gdut.edu.cn).

Recommended for acceptance by J. Wang.

Digital Object Identifier 10.1109/TETCI.2024.3500025

WITH the increasing availability of digital image editing software like Adobe Photoshop, After Effects Pro, and GIMP, images can be easily and undetectably manipulated. Image retouching operations, such as blurring, contrast enhancement, and smoothing, are widely used to enhance the visual appeal of images without affecting their semantic representation. However, content editing manipulations pose serious social risks, especially in finance, insurance, legal forensics, and journalism. The urgent task is to explore dependable models that can effectively detect manipulated images, which are often invisible to human visual inspection. Identifying subtle inconsistencies in boundaries between the original and manipulated regions can provide invaluable forensic clues for Image Manipulation Localization (IML).

IML has attracted significant attention in recent decades due to the proliferation of sophisticated forgery techniques. Historically, passive forensic methods matched the statistical properties of forged images using manually designed features such as Overlapping Patches [1], Scale-Invariant Feature Transforms (SIFT) [2], Speed-Up Robust Feature (SURF) [3], and Camera Filter Arrays (CFA) [4]. These techniques, effective against certain manipulations, are now largely inadequate for addressing the advanced composite tampering prevalent in today's image forensics. Zhang et al. [5] pioneered the application of Convolutional Neural Networks (CNNs) to forgery detection in 2016. Recently, deep neural networks (DNNs) have advanced significantly in computer vision (CV) tasks such as object detection [6], [7] and semantic segmentation [8], [9], which bear similarities to IML goals: primarily segmenting potentially tampered regions. This similarity has inspired the adaptation of CV techniques for IML, with successful implementations reported in CNN-based [10], object detection-based [11], and semantic segmentation-based [12] methods. The advent of the Vision Transformer (ViT) [13] further motivated researchers to explore its applicability to IML [14]. Despite the successes of DNNs and ViT in recognizing objects through contextual understanding, their focus may diverge from IML's unique challenges, which stem from the irregular sizes and varied shapes of forged areas, typically devoid of semantic content.

Addressing IML challenges involves several strategies: (1) extracting shallow texture features and tampering traces using

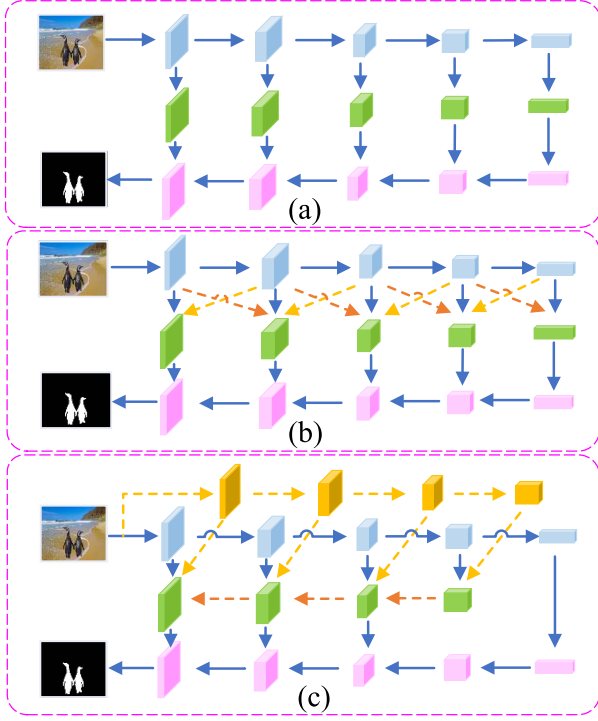


Fig. 1. Different encoding-decoding structures for IML. (a) Chen et al. [12] structure. (b) Li et al. [15] structure, and (c) Our structure.

Pre-processing Blocks [16], [17], [18]; (2) enhancing edge features of manipulated regions [14], [15], [19]; (3) implementing forensic attention mechanisms to amplify tampering traces [20], [21]; and (4) applying supervised contrastive learning to highlight feature disparities between manipulated regions and the background [22], [23]. [16], [17], [18], leveraging residual images processed through various pre-processing blocks, are particularly effective for Splicing, due to the contrast between tampered regions and the background. However, these methods show reduced sensitivity to Copy-move, where tampered regions from the same image blend seamlessly with the background. MVSS-Net [19] uses Sobel convolutions for edge discrepancy detection, while Li et al. [15] employs adaptive edge supervision based on morphological operations. EMT-Net [14] focuses on the residual strategy to highlight edge characteristics. A common shortcoming of these methods is their performance in multi-scale edge supervision, excelling on low-dimensional, high-resolution feature maps but struggling with high-dimensional, low-resolution representations. Dense prediction tasks, including semantic segmentation, instance segmentation, and IML, frequently use an encoding-decoding framework. Chen et al. [12] introduces an encoding-decoding network with lateral connections between the encoder and decoder (Fig. 1(a)). However, these connections are limited to local features from their corresponding layers and lack access to comprehensive global features. Inspired by salient object detection, Li et al. [15] develop adaptive edge supervision with a bi-directional feature fusion module to merge adjacent layer features (Fig. 1(b)). However, using only hierarchical RGB features is less effective for Copy-move tampering, as it omits texture information.

Confronted with these challenges, we are inspired to develop an efficient end-to-end network architecture for IML. Our main contributions are summarized as the following four aspects:

- 1) We propose an innovative end-to-end IML network, illustrated in Fig. 1(c), designed to accurately locate multi-scale forged regions by fusing semantic and textual features. Our network comprises three main modules: Dual-shallow Feature Pyramid Fusion (DFPF), Encoder, and Decoder. Extensive experiments across five benchmark datasets and a real-life dataset demonstrate the superior IML performance of our approach.
- 2) To the best of our knowledge, our DFPF module is the first solution to fuse features of a dual-stream feature pyramid, overcoming the traditional limitations of DNN-based IML, which lack global feature access. The DFPF integrates the RGB Feature Pyramid (RGBFP) and Local Textual Feature Pyramid (LTFP) via a bi-directional path. To effectively capture subtle manipulated traces, we introduce two Pre-processing Blocks within the LTFP branch: Parallel Multi-Scale Convolution (PMSC) and Channel Split High-frequency Convolution (CSHC).
- 3) In the Decoder module, we deploy Cascaded Boundary Contextual Incoherence Enhancement (BCIE) Blocks to enhance texture inconsistencies at the edges between the foreground and background. Each BCIE Block independently assesses pixel inconsistencies within a local window, refining the delineation of tampered regions within high-level feature representations.
- 4) Additionally, we introduce an automatic strategy for generating large-scale synthetic datasets via Photoshop Scripting¹ (PS-Scripting), which allows customization of the number, type of tampering, and post-processing to mirror real-world tampering scenarios. It ensures ample and sufficient training samples without additional manual labeling.

The paper is formed as follows: Section II presents the related work, Section III introduces our proposed method, Section IV describes the strategy of generating the synthetic datasets, Section V explains the details of experimental settings, the performance evaluations, and comparisons with existing methods. Finally, we conclude the conclusion in Section VI.

II. RELATED WORK

This section reviews the most relevant studies of DNN-based methods for IML. Then, we briefly introduce Transformer [24] used in CV, which is the encoder of our proposed approach.

A. Image Manipulation Localization

Content-changed image manipulation techniques are typically classified as Copy-Move, Splicing, and Removal. Some prior research focused on detecting and locating three types of tampering manipulations. ManTraNet [25], an LSTM-based network, converts manipulated traces into local anomalies but

¹[Online]. Available: <https://www.adobe.com/devnet/photoshop/scripting.html>

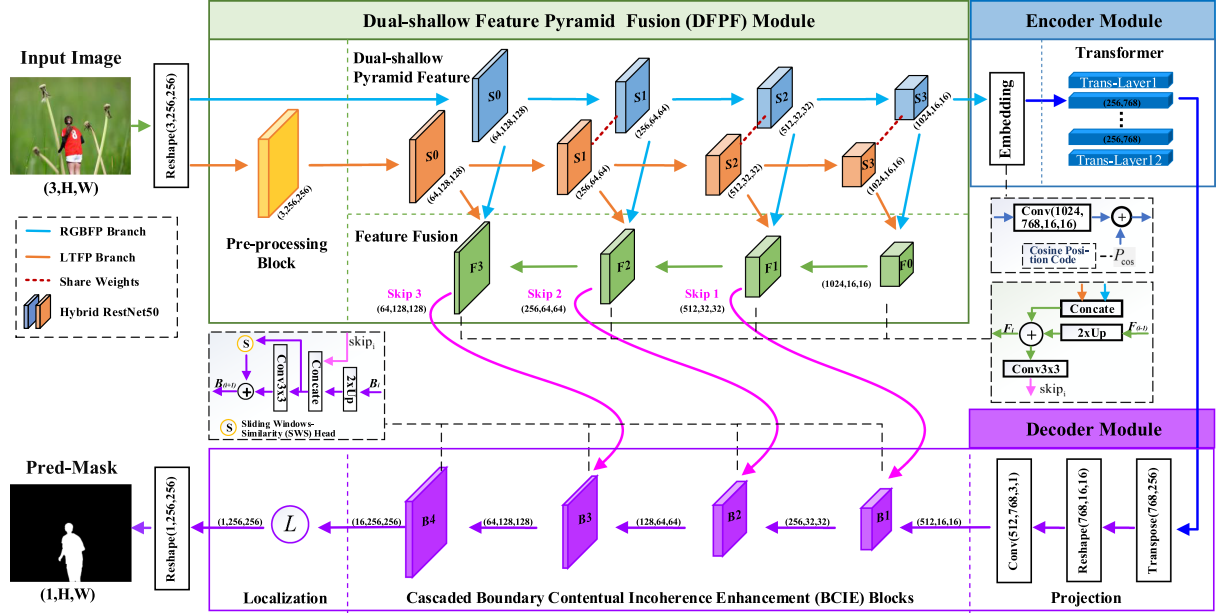


Fig. 2. Overview of the architecture. Our architecture consists of three sequential modules: DFPF, Encoder, and Decoder. The DFPF module initially extracts LTFP and RGBFP via dual Hybrid ResNet50s. The LTFP branch undergoes preprocessing with a Pre-processing Block, whereas the RGBFP branch does not. The RGB information of the input image is projected into the Encoder module via Embedding, as detailed in the corresponding dotted box. The Decoder module comprises Cascaded BCIE Blocks, which search and pinpoint inconsistencies at the pixel level in the feature map.

performs poorly. DFCN [10], a dense fully convolutional network using an encoding-decoding architecture with dilated convolutions in the decoder, introduces a strategy for generating large-scale training data via PS Scripting. EMT-Net [14] is a dual-branch network that extracts noise features via a Transformer branch and captures local RGB features through a CNN branch, emphasizing tampering traces with edge artifact enhancement modules to uncover subtle boundaries in images. However, both DFCN and EMT-Net yield unsatisfactory results. Li et al. [15] develop an adaptive edge supervision module based on morphological operations, though it is insensitive to high-dimensional features. MVSS-Net [19] proposes multi-view edge supervision to explore tampered region boundaries, using the Sobel operator in IML for the first time. These methods are not sensitive to Copy-Move tampering. TDA-Net [11] is a three-stream network based on object detection, fusing RGB, local, and resampling feature streams through ROI pooling, and employs a cross-domain attention mechanism to improve proposal efficiency. SATFL [18] is a coarse-to-fine prediction network with a forensic attention module, training on specific datasets through a self-adversarial training strategy. SAPS-Net [21] designs semantic-agnostic forgery attention to fuse pyramid features using a parallel architecture. UP-Net [20] employs two structurally identical parallel branches designed to learn tampering inconsistencies. Recently, contrastive learning techniques have been applied to IML. NCL-IML [23] introduces non-mutually exclusive contrastive learning with a dual-branch pivot structure that switches the role of contour patches in forged regions between positives and negatives. PCL [22] uses a two-stream architecture based on an object detection framework, applying supervised contrastive learning to proposals. While these methodologies are exploratory and have not yet achieved

optimal results, they provide valuable insights into unsupervised learning within IML.

B. Transformer for Computer Vision

Transformer [24], initially groundbreaking in Natural Language Processing, has been adapted for CV. ViT [13] is the first to project images into flattened-patch sequences and feed them to a Transformer encoder for image classification. SegFormer [26] introduces a hierarchically structured Transformer encoder that generates multi-scale features without positional encoding and avoids complex decoders. Maskformer [27] redefines semantic segmentation as a mask classification problem, uniformly addressing semantic and instance segmentation tasks. TransAtUnet [28] integrates Transformer self-attention with global spatial attention, significantly enhancing its ability to discern complex non-local feature interactions, thereby markedly improving the network's efficiency in recognizing intricate feature relationships. In our work, we employ ViT [13] to encode RGB information into robust, detailed flattened-patch representations.

III. PROPOSED METHOD

To accurately localize tampered regions within images, we introduce an end-to-end encoding-decoding architecture comprising three primary modules: DFPF, Encoder, and Decoder. Fig. 2 illustrates this framework, with each module distinctly colored for clarity. Initially, images are resized to $\mathbb{R}^{3 \times 256 \times 256}$. The DFPF module integrates features from dual-configured Hybrid ResNet50s [29], generating RGBFP and LTFP. The LTFP includes a Pre-processing Block that enhances texture analysis, while the RGBFP captures broader semantic features. The Encoder module converts the high-dimensional RGB features

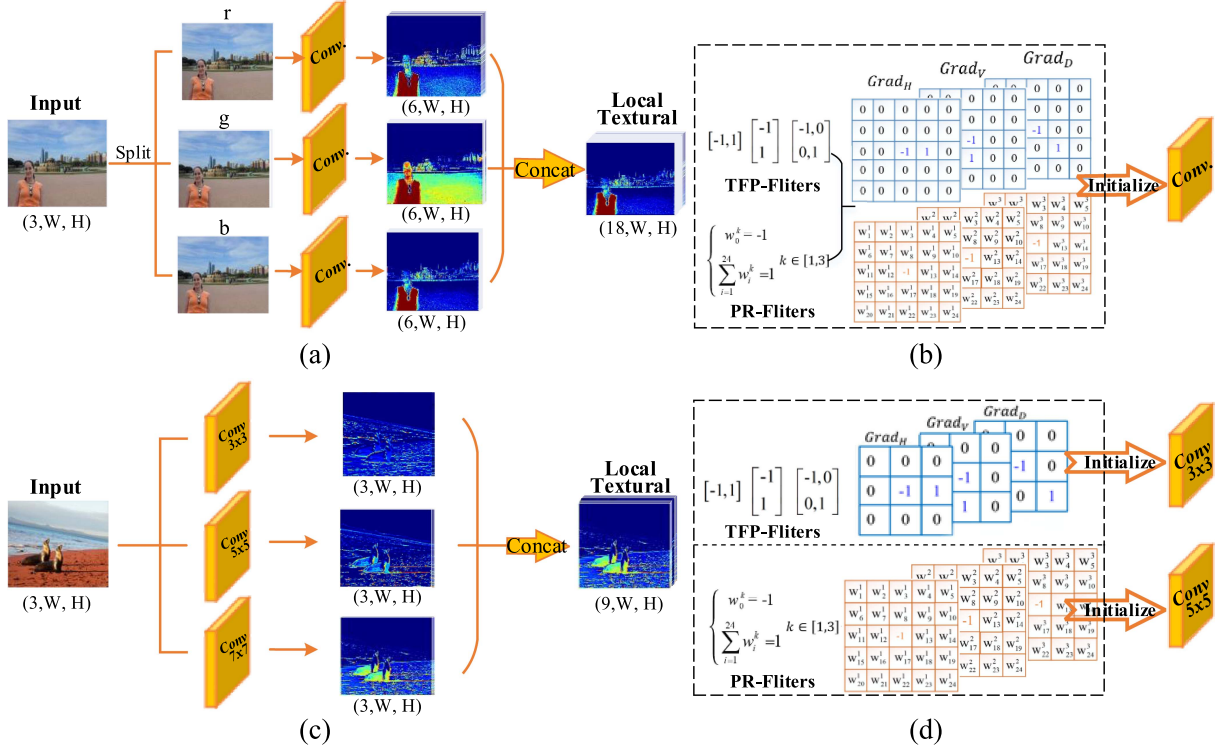


Fig. 3. Details of the Pre-processing Blocks featuring CSHC and PMSC. (a) Illustrates the operational mechanism of CSHC. (b) Shows the initial kernel configuration for CSHC, utilizing TFP Filters with a 5×5 kernel size and PR Filters, with parameters adjustable during training. (c) Demonstrates the functional mechanism of PMSC. (d) Details the initial kernel setup for PMSC: Conv3x3 uses TFP Filters at a 3×3 kernel size, Conv5x5 employs PR Filters, and Conv7x7 utilizes Kaiming Initialization.

$\mathbb{R}^{1024 \times 16 \times 16}$ into flattened-patch representations $\mathbb{R}^{256 \times 768}$. Finally, the Decoder reconstructs the image using Cascaded Boundary Contextual Incoherence Enhancement (BCIE) Blocks to predict the binary mask.

A. Dual-Shallow Feature Pyramid Fusion (DFPF) Module

The challenge of IML often arises from the non-uniform size and shape of tampered regions, which typically lack semantic content. The DFPF module integrates RGB and textual features bi-directionally, utilizing 'Siamese' Hybrid ResNet50s [29] that offer higher resolution and fewer channels than standard ResNet50 [29]. In the forward path, dual parallel Hybrid ResNet50s [29] serve as the backbone, extracting two shallow feature pyramids: the RGBFP and LTFP. Conversely, in the reverse path, Feature Fusion blocks generate new multi-scale features F_i ($i = 0, 1, 2, 3$) by amalgamating RGB and textual features at corresponding levels. Compared to [15], the DFPF module delivers a shallower architecture with higher-resolution features and richer global information on manipulated traces by combining RGB and textual features.

Pre-processing Block: Current IML methods utilize various static filters like SRM [16], CW-HPF [18], and Bayar [17]. SRM and CW-HPF, being manually designed and unlearnable, are susceptible to manipulation. Bayar filters offer learnable kernels for residual noise prediction but are constrained by fixed central kernel values. To improve the detection of tampered traces and

contour accuracy, we introduce a novel pre-processing strategy that integrates a Tri-directional First-order Partial-derivative Filter (TFP Filter) with a Predicted Residual Filter (PR Filter). The TFP Filter, derived from the 1D mask approximations stated in (1) and (2), calculates horizontal, vertical, and diagonal partial derivatives to refine contour details and is initialized with three gradient operators: $Grad_H$, $Grad_V$, and $Grad_D$, shown as Fig. 3(b) and 3(d). These operators enable precise initial processing, with subsequent supervised learning adjustments during back-propagation to improve tampered area localization. Additionally, the PR Filter, informed by the predicted residual concept (3), dynamically estimates pixel residual values from their neighbors. It sets the central value (W_0^k) to -1, with other weights summing to 1, detailed as (4), thus facilitating an adaptive response to varying image manipulations. This dual-filter approach significantly boosts the accuracy of manipulation localization.

$$g_x = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y) \quad (1)$$

$$g_y = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y) \quad (2)$$

$$r = \mathcal{P}(\mathbf{I}) - \mathbf{I} \quad (3)$$

$$\begin{cases} W_0^k = -1 \\ \sum_{i=1}^{24} W_i^k = 1 \end{cases} \quad (4)$$

Where g_x and g_y represent the horizontal and vertical first derivatives, respectively, while r denotes the predicted residual of the input image I . The term W_0^k signifies the central value of the PR Filter, where k indicates the kernel's channel. Additionally, $\mathcal{P}(\cdot)$ symbolizes a 2D convolution operation, and the PR Filter is configured with a kernel size of 5.

Our methodology introduces two innovative Pre-processing Blocks: CSHC and PMSC. As illustrated in Fig. 3(a) and 3(b), CSHC processes RGB channels of the image $I \in \mathbb{R}^{3 \times H \times W}$ individually, applying channel-wise convolutions that produce concatenated feature maps of size $\mathbb{R}^{18 \times H \times W}$. This enhances inter-channel contrast and leverages TFP and PR Filters to improve tampering detection. PMSC employs three parallel convolution layers with varying kernel sizes $\{3 \times 3, 5 \times 5, 7 \times 7\}$ to broaden the receptive field, as depicted in Fig. 3(c) and 3(d). While the Conv3x3 layer incorporates the TFP Filter, the Conv5x5 utilizes the PR Filter. Enhancing parameter efficiency, the Conv7x7 layer is replaced by two sequential Conv3x3 layers with Batch Normalization and Swish Activation. The outputs from these layers are combined to form a feature map of $\mathbb{R}^{9 \times H \times W}$, significantly boosting the model's ability to localize manipulated regions precisely.

Hybrid ResNet50: We adjust the sizes and channels of the output at each stage of the standard ResNet50 [29] to fit our task, maintaining the 50 convolution layers. Specifically, the output $s_0 \in \mathbb{R}^{64 \times 128 \times 128}$ from the Stem Layer before Maxpool2D in the Hybrid ResNet50 is considered the first feature map. The three stages stack along $\{3, 4, 9\}$ Bottleneck Layers, compared to standard ResNet50 [29]. The output sizes of three stages are $\{\frac{H}{4} \times \frac{W}{4}, \frac{H}{8} \times \frac{W}{8}, \frac{H}{16} \times \frac{W}{16}\}$, reduced by half per stage. The channels follow the increasing rule of $\{256, 512, 1024\}$. Unlike the standard ResNet50 [29], the feature pyramid extracted using Hybrid ResNet50 [29] exhibits higher resolutions with shallower channels.

Feature Fusion: Fig. 2 represents the reverse pathway within the DFPF module, two coarse-resolution feature maps $s_3 \in \mathbb{R}^{1024 \times 16 \times 16}$ from the RGBFP and LTFP branches are concatenated to initiate the fused feature $F_0 \in \mathbb{R}^{1024 \times 16 \times 16}$. This feature is upsampled and merged with the adjacent layers of RGBFP and LTFP via element-wise addition, generating an advanced fused feature $F_1 \in \mathbb{R}^{512 \times 32 \times 32}$. This upsampling and fusion continue until the highest resolution map $F_3 \in \mathbb{R}^{64 \times 128 \times 128}$ is attained. The resulting outputs from the Feature Fusion blocks form a reverse fusion feature pyramid, progressively integrating features from identical and subsequent layers, enhancing global feature representation. Each fusion level acts as a skip connection, augmenting textural details for the Cascaded BCIE Blocks. A Conv3x3 operation is applied to these fused features to reduce the aliasing effect of upsampling, optimizing skipping feature maps $\{Skip_1, Skip_2, Skip_3\}$. The detailed structure of Feature Fusion is elaborated in (5).

$$\begin{cases} F_l = \text{Conv}_{3 \times 3} \left(\text{up}(F_{l-1}) + \text{Conv}_{1 \times 1}(\sigma(s_{3-l}^r, s_{3-l}^f)) \right) \\ F_0 = \text{Conv}_{1 \times 1}(\sigma(s_3^r, s_3^f)), \quad l = 1, 2, 3. \end{cases} \quad (5)$$

Where subscript l denotes the l -th Feature Fusion block. F_l is the output $Skip_l$ from Feature Fusion, with s_l^r and s_l^f indicating the inputs from RGBFP and LTFP, respectively. The operation $\sigma(\cdot)$ represents concatenation, while up denotes upsampling by a factor of 2.

B. Transformer as Encoder Module

Due to their inherent designs, CNNs excel at extracting features from local receptive fields, while Transformers effectively capture long-range dependencies. To leverage these strengths, we develop a ViT-based Encoder module that encodes high-dimensional RGBFP globally. Initially, RGB features $s_3 \in \mathbb{R}^{1024 \times 16 \times 16}$ are reshaped into n vectorized patches of size $\{16 \times 16\}$, making the length of each patch 768. These are then transformed to $s'_3 \in \mathbb{R}^{768 \times 16 \times 16}$ using $\text{Conv}_{1 \times 1}$ operations. Subsequently, s'_3 is embedded into 2D sequences without the class token, resulting in linear patch projections $P_0 \in \mathbb{R}^{256 \times 768}$ after transposition. This process, which deviates from standard ViT [13] sequentialization, incorporates learnable positional codes P_{cos} to maintain spatial information of tampered regions. The embedding process is explained in (6). Following this, the enhanced global representation capabilities of ViT prompt us to feed P_0 into 12 Transformer Layers comprising Multi-head Self-Attention (MSA) and Multi-Layer Perceptron (MLP) blocks. The output of the l_{th} Transformer Layer is described in (7).

$$P_0 = \mathcal{T}(\text{Conv}_{1 \times 1}(s_3)) + P_{cos} \quad (6)$$

$$\begin{cases} P'_l = \text{MSA}(\text{LN}(P_{l-1})) + P_{l-1} \\ P_l = \text{MLP}(\text{LN}(P'_l)) + P'_l \quad l \in [1, 12] \end{cases} \quad (7)$$

Where P_0 represents the input to the Transformer, $\mathcal{T}(\cdot)$ indicates the transposition operation, and $P_{cos} \in \mathbb{R}^{256 \times 768}$ denotes the cosine positional coding. $\text{LN}(\cdot)$ represents the layer normalization operator and P_l is the encoded feature representation of the l_{th} Transformer Layer.

C. Cascaded Boundary Contextual Incoherence Enhancement (BCIE) Blocks

In the Decoder module, the encoded features $P_{12} \in \mathbb{R}^{\frac{H \times W}{P^2} \times D}$ are processed into $x' \in \mathbb{R}^{512 \times \frac{W}{24} \times \frac{H}{24}}$ using a series of Transpose, Reshape, and $\text{Conv}_{3 \times 3 \text{BnRelu}}$ operations. These operations are critical in IML for highlighting edge incoherence between tampered regions and their backgrounds. To enhance the incoherence in the final binary mask prediction, we introduce Cascaded BCIE Blocks in the Decoder. These blocks intensify boundary inconsistencies and incorporate lateral connections $\{Skip_i | i = 1, 2, 3\}$ from the DFPF module, enriching textural and locational details. Each BCIE Block follows a sequence of operations: $2 \times \text{Upsampling}$, Concatenation, $\text{Conv}_{3 \times 3 \text{BnRelu}}$, and integrates a Sliding Window Similarity Head (SWS-Head). The SWS-Head calculates the mean cosine similarity between each pixel and its neighbors within a local window, refining boundary detection as outlined in Fig. 2 and detailed in (8). This method effectively delineates tampered edges by distinguishing 'hard positive' and 'hard negative' pixels, which exhibit distinct

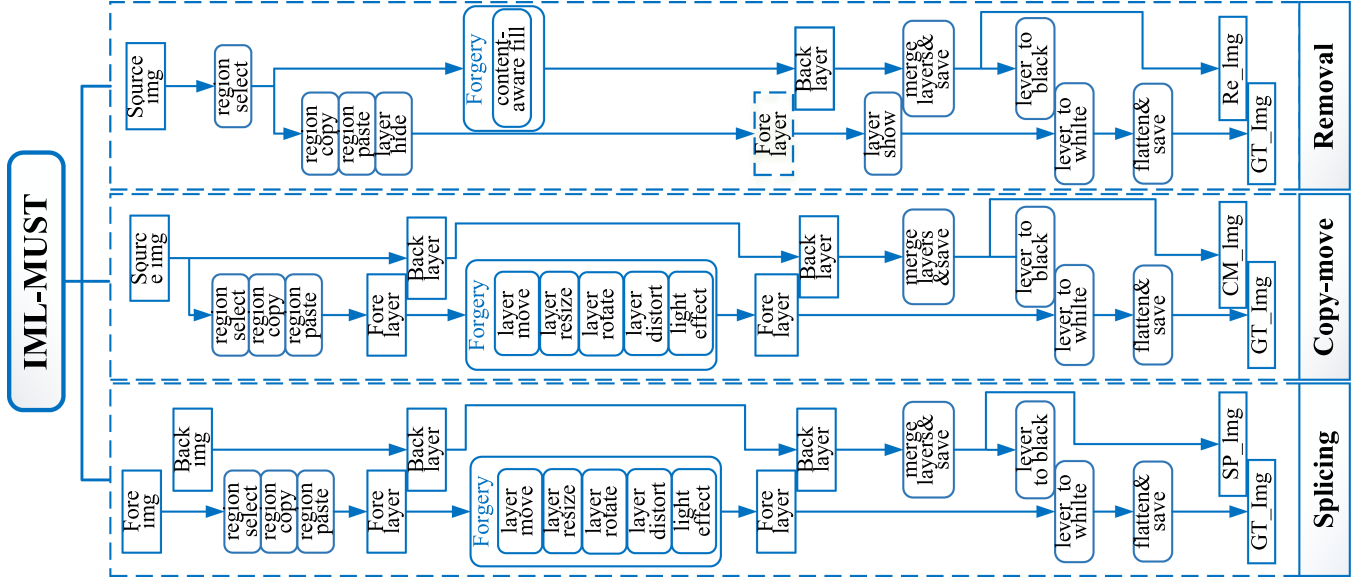


Fig. 4. Procedure of IML-MUST dataset generation.

similarity scores compared to adjacent regions. To improve the precision of tampered area localization, the similarity matrix is merged into the decoded feature map via element addition, enhancing the visibility of boundary discrepancies in the final image analysis. Our experiments show that the similarity matrix significantly improves the clarity of boundary delineation in tampered regions.

$$y_{(h,w)} = \frac{1}{K \times K} \sum_{i,j} Sim(x_{(h,w)}, x_{(h+i,w+j)})$$

$$i, j \in \left[-\frac{K-1}{2}, \frac{K-1}{2} \right] \quad (8)$$

Where $y_{(h,w)}$ denotes as an element of the Similarity Matrix (SM), the subscript (h, w) means the row and column positions of SM, $Sim(\cdot)$ represents the function of the cosine similarity operator, K is the size of the local sliding window. According to experimental results, we set $K = 3$ in this paper.

IV. STRATEGY OF SYNTHETIC DATA GENERATION

Training a high-quality model for IML necessitates a large and diverse dataset. However, creating such a dataset is labor-intensive, requiring meticulous image manipulation and annotation. To overcome this challenge, we develop a PS-Scripting-based synthesis strategy to automatically generate tampered images on a large scale, as illustrated in Fig. 4. This process includes Copy-move, Splicing, and Removal manipulations. For Splicing, two images are randomly selected, one as the background and the other as the donor. Copy-move and Removal manipulations use a single image. The foreground regions are identified using Label-me,² and transformations such as distortion, rotation, and scaling are applied to simulate human-like

manipulations. These regions are then reinserted into the background image, with additional edits like blurring and illumination adjustments to disguise the manipulations further. Our methodology permits customization of synthetic dataset parameters, such as quantity, type of tampering, and image format, making it adaptable to different tampering techniques. The original images are sourced from VISION [30], KCMI [31], and our photograph collection. Following this strategy, we synthesize the IML-MUST dataset, which includes diverse tampered samples and their ground truths. Unlike the PS-boundary dataset [10], which mainly features rectangular tampered areas, IML-MUST includes Removal manipulations and offers greater flexibility and realism, establishing it as a superior resource for training and evaluating IML models.

V. EXPERIMENTS AND ANALYSIS

A. Datasets

Our model undergoes comparative evaluation using five benchmark datasets and a real-life dataset to assess its performance comprehensively. CASIA [32] includes 920 tampered samples in v1.0 and 5,123 in v2.0, with v1.0 serving as our test set and a subset of 2324 samples from v2.0 used for training. Coverage [33] comprises 100 copy-move tampered images, utilizing a 75%-25% training-testing split. NIST2016 [34] features 564 diverse forgery samples, adhering to a 414:150 training-testing split, in alignment with Liu et al. [15]. Columbia [35] contains 180 splicing images, split into 130 for training and 50 for testing, consistent with Liu et al. [15]. IMD2020 [36] consists of 2,010 real-life manipulated images, distributed into 70% training, 5% validation, and 25% testing phases following UP-Net [20]. DEFACTO [37] is a large, pivotal dataset for image forensics, with a randomly sampled subset of 80,000 images split into an 8:1:1 training-validation-testing ratio. Lastly, our synthetic IML-MUST dataset includes 125,500 images, with

²[Online]. Available: <https://github.com/labelmeai/labelme.git>

90% designated for training and 10% for validation, covering a variety of manipulations.

B. Implementation Details

Our training utilizes three NVIDIA GeForce 3090 GPUs with the PyTorch framework. We employ the Adam [38] optimizer with a linear learning rate (as specified in (9)), a batch size of 21, and conduct training over 70 epochs. To accommodate the patch size of ViT, all images are resized to 256×256 pixels. Training consists of two stages: initial pre-training on IML-MUST or DEFACTO [37], followed by fine-tuning on a composite set from five benchmark datasets [32], [33], [34], [35], [36]. Pre-training is initialized using the Hybrid ResNet50 [29] and ViT [13] models pre-trained on ImageNet. The model employs an equal combination of Binary Cross-Entropy and Dice Loss. Performance evaluation is conducted using pixel-level F1 scores and AUC metrics, which are critical for accurately locating tampered regions. A fixed F1 threshold of 0.5 facilitates fair comparison with competing models.

$$lr = lr_0 \times \left(1.0 - \frac{iter}{Iter_{\max}}\right)^{0.9}, \quad iter \in [1, Iter_{\max}] \quad (9)$$

Where lr stands for learning rate, lr_0 denotes the initial learning rate, which is set to $1e-4$, $Iter_{\max}$ is the maximum iteration during training, $iter$ is the number of iterations, ranging from 1 to $Iter_{\max}$.

C. Ablation Analysis

Our model employs the DFPF module and Cascaded BCIE Blocks to enhance performance. Extensive experiments assess their impact on the IML-MUST dataset and evaluate F1 scores across the full set of four benchmark datasets: CASIA v1.0 [32], Coverage [33], NIST2016 [34], and Columbia [35]. The lack of overlap between training and test sets during pre-training ensures that F1 scores reliably measure performance. The settings of all ablation experiments, including batch size, epochs, optimizer, learning rate, loss function, and model initialization, are consistent with the pre-training setup.

Ablation study for DFPF: Our comprehensive evaluation of the DFPF module introduces progressive variants through various testing protocols. Initially, the baseline model ('w/o DFPF') only uses the Encoder and Decoder, excluding feature fusion. Subsequent configurations add complexity: 'w RGBFP' included a CNN for the RGB feature pyramid; 'w LTFP' integrates a Pre-processing Block based on 'w RGBFP' configuration; and 'w DFPF' employs the entire DFPF module for advanced feature fusion, significantly enhancing performance by the average F1 of 22.7% over the baseline, as detailed in Table I. This module outperforms those relying solely on RGBFP or LTFP as architected in Fig. 1(b), marking a substantial enhancement in performance.

Table II details an ablation study on the DFPF module, evaluating various pre-processing techniques. The baseline model uses dual RGB feature branches without Pre-processing Blocks. Adding convolution ('w Conv2D') modestly improves average F1 scores by 0.7%. Other setups like adding CW-HPF ('w CW-HPF') or Bayar ('w Bayar') show limited enhancements.

TABLE I
F1 SCORES OF ABLATION STUDY FOR DFPF MODULE. THE BASELINE MODEL ('w/o DFPF') CONSISTS ONLY OF THE ENCODER AND DECODER

Components	Datasets				Average-F1
	Columbia	CASIA	NIST2016	Coverage	
w/o DFPF	0.438	0.242	0.306	0.301	0.322
w RGBFP	0.698	0.537	0.395	0.479	0.527
w LTFP (CSHC)	0.699	0.443	0.330	0.476	0.487
w LTFP (PMSC)	0.718	0.419	0.356	0.462	0.489
w DFPF (CSHC)	0.738	0.534	0.385	<u>0.502</u>	0.540
w DFPF (PMSC)	<u>0.727</u>	<u>0.535</u>	0.406	0.528	0.549

The baseline model ('w/o DFPF') consists only of the encoder and decoder. The 'w RGBFP' configuration introduces a CNN branch as a skip connection supplement. 'w LTFP' builds on 'w RGBFP' by adding a pre-processing block, and 'w DFPF' incorporates the entire DFPF module. Parentheses specify the pre-processing block used in each configuration. The best and second-best results are respectively highlighted in bold and underlined to emphasize performance variations.

TABLE II
F1 SCORES OF ABLATION STUDY FOR DFPF MODULE WITH DIFFERENT PRE-PROCESSING BLOCKS

Components	Datasets				Average-F1
	Columbia	CASIA	NIST2016	Coverage	
w/o Pre-pro	0.708	0.525	0.379	0.492	0.526
w Conv2D	0.691	0.542	0.371	0.526	0.533
w CW-HPF	0.692	0.510	0.363	0.496	0.515
w Bayar	0.701	0.529	0.360	0.529	0.530
w CSHC	0.738	0.534	<u>0.385</u>	0.502	0.540
w PMSC	<u>0.727</u>	<u>0.535</u>	0.406	<u>0.528</u>	0.549

The configurations include: 'w/o pre-pro' (no pre-processing), and strategies employing various pre-processing blocks such as 'w conv2D', 'w Bayar', 'w CW-HPF', 'w CSHC', and 'w PMSC'. The best outcomes are highlighted in bold, and the second-best results are underlined for emphasis.

TABLE III
F1 SCORES OF ABLATION STUDY WITH CASCADED BCIE BLOCKS

Components	Datasets				Average-F1
	Columbia	CASIA	NIST2016	Coverage	
w/o SWS-Head(CSHC)	0.681	0.554	0.398	0.477	0.528
w/o SWS-Head(PMSC)	0.708	0.529	<u>0.391</u>	0.494	0.531
w CBAM(CSHC)	0.726	0.535	0.380	0.501	0.536
w CBAM(PMSC)	0.720	<u>0.547</u>	0.375	<u>0.505</u>	0.537
w SWS-Head(CSHC)	0.738	0.534	0.385	0.502	0.540
w SWS-Head(PMSC)	<u>0.727</u>	0.535	0.406	0.528	0.549

The configurations include: 'w/o SWS-Head' (cascaded upsampling without SWS-Head), 'w CBAM' (cascaded upsampling with CBAM), and 'w SWS-Head' (cascaded upsampling with SWS-Head). Parentheses indicate the specific pre-processing block utilized in each model configuration. The best outcomes are indicated in bold, while the second-best outcomes are underlined for emphasis.

Notably, the CSHC filter significantly improves performance in the Columbia [35] and CASIA [32], raising average F1 scores to 0.540. The PMSC strategy excels in refining tampered area detection with its varied kernel sizes.

Ablation study for Cascaded BCIE Blocks: Table III discusses the impact of the Cascaded BCIE Blocks on our framework. Comparing the baseline model's cascaded upsampling to setups incorporating edge enhancement techniques like SWS-Head or CBAM shows varied improvements. The average F1 scores are 0.528 and 0.531 for the CSHC and PMSC Pre-processing variants of the baseline, respectively. There is a slight improvement when cascaded upsampling is supplemented with the CBAM component ('w CBAM'). The integration of SWS-Head notably increased F1 scores to 0.540 for CSHC and 0.549 for PMSC,

TABLE IV
COMPARISON OF AUC SCORES ACROSS FIVE BENCHMARK DATASETS WITH VARIOUS MODELS

Methods	Datasets					Ave-AUC
	Columbia	CASIA	NIST2016	Coverage	IMD2020	
ManTraNet [25]	0.824	0.817	0.795	0.819	-	0.814
TDA-Net [11]	0.892	0.831	0.948	0.864	-	0.884
SATFL [18]	0.917	0.788	0.943	0.856	-	0.876
EMT-Net [14]	0.832	0.856	0.987	0.812	-	0.872
Li et.al [15]	0.962	0.701	0.954	-	-	0.872
SAPS-Net [21]	-	0.843	0.996	0.852	-	0.897
NCL-IML [23]	0.943	-	0.912	0.928	-	0.928
UP-Net [20]	-	-	0.995	0.879	0.908	0.927
PCL [22]	0.761	0.751	0.946	0.917	0.823	0.840
OURS	0.975	0.863	0.983	0.947	<u>0.942</u>	0.942
OURS+	0.975	0.863	0.977	<u>0.939</u>	0.950	<u>0.941</u>

A dash ('-') indicates that the results are unavailable in the paper and code. The best and second-best scores are highlighted in bold and underlined, respectively. 'Ave-AUC' represents average AUC scores. 'OURS' and 'OURS+' denote models respectively enhanced with CSHC and PMSC, while pre-trained on IML-must.

TABLE V
COMPARISON OF F1 SCORES ACROSS FIVE BENCHMARK DATASETS FOR VARIOUS MODELS

Methods	Datasets					Ave-F1
	Columbia	CASIA	NIST2016	Coverage	IMD2020	
MVSS-Net [19]	0.660	0.513	0.304	0.482	0.270	0.421
TDA-Net [11]	0.735	0.582	0.756	0.474	-	0.637
SATFL [18]	0.891	0.384	0.622	0.562	-	0.615
EMT-Net [14]	0.561	0.459	0.825	0.353	-	0.550
Li et.al [15]	0.833	0.456	0.854	-	-	0.714
SAPS-Net [21]	-	<u>0.561</u>	0.878	0.507	-	0.649
NCL-IML [23]	0.850	-	0.831	0.801	-	0.621
UP-Net [20]	-	-	0.916	0.577	<u>0.574</u>	0.689
PCL [22]	0.698	0.467	0.780	0.620	0.503	0.614
OURS	0.930	0.539	0.869	0.683	0.553	<u>0.715</u>
OURS+	0.947	0.547	0.880	0.686	0.600	0.732

A dash ('-') indicates that the results are unavailable in the paper and code. The best and second-best scores are highlighted in bold and underlined, respectively. 'Ave-F1' represents the average F1 scores. 'OURS' and 'OURS+' denote models enhanced with CSHC and PMSC, respectively, while pre-trained on IML-MUST. MVSS-Net is pre-trained on CASIA v2.0, which represents the optimal weight.

demonstrating its efficacy in enhancing tampered region localization, especially in Columbia [35] and NIST2016 [34]. This performance boost is credited to SWS-Head's ability to identify edge inconsistencies effectively, thus minimizing false positives associated with upsampling.

D. Comparison With the States-of-The-Art Methods

Our method competes against leading deep learning algorithms in IML, including ManTraNet [25], MVSS-Net [19], TDA-Net [11], SATFL [18], EMT-Net [14], Li et al. [15], SAPS-Net [21], NCL-IML [23], UP-Net [20], and PCL [22]. Section II-A provides a comprehensive overview of each algorithm. We evaluate our approach on five benchmark datasets and a real-life dataset: CASIA [32], Coverage [33], NIST2016 [34], Columbia [35], DEFACTO [37], and IMD2020 [36], using pixel-level AUC and F1 scores to compare predictions against ground truths.

Tables IV and V present the results of evaluation experiments using fine-tuned weights on a combined training set from [32], [33], [34], [35], [36]. The fine-tuned model is initialized with the best pre-training weights from IML-MUST. Table IV confirms our method's superiority across four datasets [32], [33], [35], [36] on AUC scores, with significant improvements in

TABLE VI
COMPARISON OF F1 AND AUC SCORES ACROSS DEFACTO AND IMD2020 FOR VARIOUS MODELS

Methods	Training Set	DEFACTO		IMD2020	
		F1	AUC	F1	AUC
MVSS-Net* ⁴ [19]	DEFACTO	0.553	0.786	0.066	0.530
SATFL ⁵ [18]	DEFACTO	0.080	0.654	0.139	0.616
INC-IML ⁶ [23]	DEFACTO	0.607	0.889	0.156	0.574
OURS*	DEFACTO	0.806	0.991	0.124	<u>0.646</u>
OURS*+	DEFACTO	0.806	<u>0.989</u>	<u>0.146</u>	0.709

A dash ('-') indicates that the results are unavailable in the paper and the code. The best and second-best scores are highlighted in bold and underlined, respectively. 'OURS' and 'OURS+' denote models respectively enhanced with CSHC and PMSC, while pre-trained on DEFACTO. MVSS-Net* is specifically pre-trained on DEFACTO.

Columbia [35] ranging from 1.3% to 21.4%. Although our method ranks second on NIST2016 [34], it closely approaches the top performer. Table V highlights our method's superior average F1 scores, excelling particularly in Columbia [35] and IMD2020 [36], and achieving second-best in CASIA [32] and NIST2016 [34]. We exclude NCL-IML [23] and UP-Net [20] from our CASIA [32] analysis, as they are trained on a CASIA [32] v1.0 subset not used in our training, making direct comparisons inappropriate. Our model slightly underperforms compared to UP-Net [20] on NIST2016 [34], is less effective than TDA-Net [11] and SAPS-Net [21] on CASIA [32], and marginally inferior to NCL-IML [23] on Coverage [33]. Consistent with established protocols [14], [15], [18], [19], [25], we fix the F1 threshold at 0.5, which differs from [11], [20], [21], [23]. Adjusting to an optimal threshold significantly improves our F1 scores. Testing the official NCL-IML [23] code with a 0.5 threshold reveals lower F1 scores for Coverage [33] than reported, indicating discrepancies. TDA-Net [11], UP-Net [20], and SAPS-Net [21] could not be tested due to unavailable code.

Our study encounters challenges due to the absence of IMD2020 [36] test results for many models. To address this, we use officially published weights from GitHub for a comprehensive evaluation of IMD2020 [36], focusing on models such as MVSS-Net [19], SATFL [18], and NCL-IML [23]. We use the full set of IMD2020 [36] for testing with pre-trained weights on DEFACTO [37], ensuring the testing set remains uncontaminated by training data. Results detail in Table VI confirm our model's superiority in real-life forgery scenarios. This extensive evaluation underscores the robustness of our model and its efficacy in managing diverse datasets.

The computational complexities are calculated and compared to existing benchmarks using 256×256 NIST2016 [34] forged images on a single NVIDIA GeForce RTX 3090 GPU. Training time refers to the total duration for 10 epochs on NIST2016 [34], utilizing identical settings, including data augmentation, loss function, learning rate, and optimizer. Inference time is the average time tested on competitors' public models using CASIA v1.0 [32]. For benchmarking, we compare our method to NCL-IML [23] and MVSS-Net [19], as shown in Table VII. Because NCL-IML [23] only released the best weights, we could not reproduce their work and calculate training time accurately. Our method demonstrates acceptable model parameters, GFLOPs, and comparable training times. With PMCS, our method takes about 115.3 ms per image for evaluation, which is slightly slower

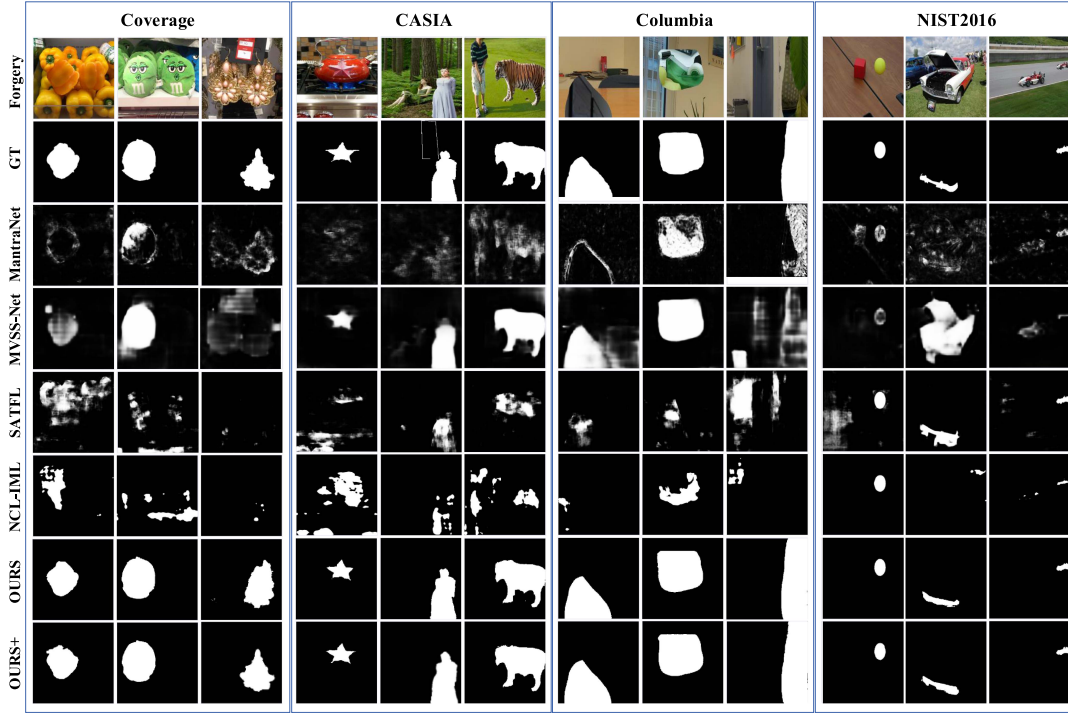


Fig. 5. Visualization of the detected results by different methods. From top to bottom, the images include the forged images, GT labels, results from ManTraNet [25], MVSS-Net [19], SATFL [18], NCL-IML [23], OURS, and OURS⁺. 'OURS' and 'OURS⁺' denote models respectively enhanced with CSHC and PMSC, while pre-trained on IML-MUST.

TABLE VII
COMPUTATIONAL COMPLEXITY COMPARED WITH
STATE-OF-THE-ART METHODS

Methods	Parames(M)	GFLOPs	Interence Time(ms)	Training Time(m)
MVSS-Net [19]	142.78	40.89	82.66	6.31
NCL-IML [23]	59.34	65.27	85.41	-
OURS	101.12	42.47	103.30	6.71
OURS ⁺	101.12	42.49	115.39	6.35

'M', 'ms', and 'm' represent millions, milliseconds, and minutes. A dash ('-') indicates the results are unavailable in the paper and code. 'OURS' and 'OURS⁺' denote models respectively enhanced with CSHC and PMSC, while pre-trained on IML-MUST.

but still practical. This slight increase in computational complexity is due to the DFPF. However, ablation analysis (Table I) shows that DFPF significantly enhances model performance. Overall, our method's efficiency is acceptable.

E. Visualization

In this section, we present a visual comparison between our approach and three competitors: ManTraNet³ [25], MVSS-Net⁴ [19], SATFL⁵ [18], and NCL-IML⁶ [23] across four benchmark datasets: CASIA v1.0 [32], Coverage [33], NIST2016 [34], and Columbia [35], with results depicted in Fig. 5. Notably, all binary masks are stored directly from the model's predictions using the Pillow API, bypassing any morphological or edge

enhancements. Our method consistently surpasses these competitors in manipulation localization accuracy, particularly in Coverage [33], where it adeptly identifies Copy-move forgery, a common challenge for other models. As seen in columns 10 to 12 of Fig. 5, our model also outperforms competitors in localizing small tampered areas. Competitors often overlook essential global features critical for IML. Our model employs the DFPF module and Cascaded BCIE Blocks to enhance the detection of edge inconsistencies and tampered boundaries.

Furthermore, an edge enhancement mechanism leveraging Cosine Similarity within the Cascaded BCIE Blocks improves decoding accuracy. Grad-CAM visualizations (Fig. 6) demonstrate our model's capability in pinpointing tampered regions and accentuating texture inconsistencies along the borders across various manipulation scenarios.

F. Robustness Evaluation

Table VIII provides the robustness analysis of the models, showing pixel-level AUC results. For a fair comparison, we use the same strategy as SPAN [39] and NCL-IML [23], employing OpenCV built-in functions to attack NIST2016 [34] dataset. The attacks include LinearResize, GaussianBlur, GaussianNoise, and JPEGCompress. Compared to SPAN [39] and NCL-IML [23], our model consistently achieves higher AUC scores across various attacks, demonstrating superior robustness. Notably, our model shows strong resilience to GaussianBlur, GaussianNoise, and JPEGCompress attacks, with a 0.3% increase under 'GaussianBlur (size=15)'. We attribute this to

³[Online]. Available: <https://github.com/ISICV/ManTraNet.git>

⁴[Online]. Available: <https://github.com/dong03/MVSS-Net.git>

⁵[Online]. Available: <https://github.com/tansq/SATFL.git>

⁶[Online]. Available: <https://github.com/Knightzjz/NCL-IML.git>

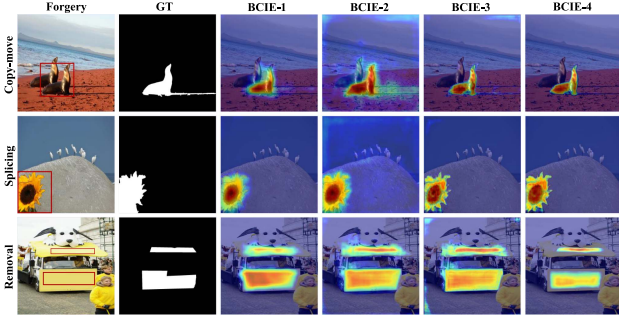


Fig. 6. Visualization of Grad-CAM features of three examples from Cascaded BCIE Blocks in our proposed model enhanced with CSHC, pre-trained on IML-MUST (all from NIST2016 [34]). The 1st column displays the forged samples, where the tampered area is highlighted in a red rectangle. As more BCIE Blocks are cascaded, the focus on the tampered areas becomes more precise. The Grad-CAM results from BCIE-4 demonstrate accurate localization of the forged regions.

TABLE VIII
ROBUSTNESS EVALUATION OF MODELS ON NIST2016 DATASET

Operations	SPAN [39]	NCL-IML [23]	OURS	OURS ⁺
None	0.836	0.912	0.983	0.975
Resize(0.78x)	0.832	0.856	0.981	0.971
Resize(0.25x)	0.803	0.831	0.969	0.959
GaussianBlur(size=3)	0.831	0.840	0.982	0.973
GaussianBlur(size=5)	-	0.806	0.984	0.977
GaussianBlur(size=15)	0.792	-	0.986	0.975
GaussianNoise(size=3)	0.751	0.795	0.981	0.973
GaussianNoise(size=5)	-	0.714	0.981	0.970
GaussianNoise(size=15)	0.673	-	0.950	0.959
JPEGCompress(100)	0.836	0.819	0.981	0.975
JPEGCompress(50)	0.807	0.843	0.981	0.972

The results are reported in pixel-level AUC. A dash ('-') indicates the results are unavailable in the paper. The best and second-best scores are highlighted in bold and underlined, respectively. 'OURS' and 'OURS+' denote models respectively enhanced with CSHC and PMSC, while pre-trained on IML-MUST.

two main factors: 1) Cascaded BCIE Blocks enhance tampered edge artifacts, and 2) DFPF excels in extracting and integrating rich tampered information.

VI. CONCLUSION

Our research introduces a cutting-edge end-to-end network tailored to pixel-level localization in IML, featuring an innovative encoding-decoding architecture. The pivotal component of our network, the DFPF module, skillfully combines RGBFP and LTFP features, enhancing global and local texture information for the decoding process. In the Encoder, a Transformer converts RGB features into flattened-patch sequences, ensuring robust global representations. Meanwhile, the Decoder's Cascaded BCIE Blocks address texture inconsistencies, effectively harnessing low-level tampering trace features for accurate localization. Experimental evidence demonstrates our model's superior performance in IML across diverse datasets, highlighting its innovation. However, the model is sensitive to forgery traces, performing optimally when training and test datasets involve the same tampering techniques, yet it underperforms with unfamiliar manipulations. This underscores the necessity of training on diverse datasets to improve generalization. These findings will guide the focus of our future research.

REFERENCES

- [1] H. Farid and S. Lyu, "Higher-order wavelet statistics and their application to digital forensics," in *Proc. 2003 Conf. Comput. Vis. Pattern Recognit. Workshop*, 2003, vol. 8, pp. 94–94.
- [2] H. A. Alberry, A. A. Hegazy, and G. I. Salama, "A fast sift based method for copy move forgery detection," *Future Comput. Informat. J.*, vol. 3, no. 2, pp. 159–165, 2018.
- [3] R. C. Pandey, S. K. Singh, K. K. Shukla, and R. Agrawal, "Fast and robust passive copy-move forgery detection using surf and sift image features," in *Proc. 2014 9th Int. Conf. Ind. Inf. Syst.*, 2014, pp. 1–6.
- [4] P. Ferrara, T. Bianchi, A. De Rosa, and A. Piva, "Image forgery localization via fine-grained analysis of CFA artifacts," *IEEE Trans. Inf. Forensics Secur.*, vol. 7, no. 5, pp. 1566–1577, Oct. 2012.
- [5] Y. Zhang, J. Goh, L. L. Win, and V. L. Thing, "Image region forgery detection: A deep learning approach," *SG-CRC*, vol. 2016, pp. 1–11, 2016.
- [6] W. Zhou, Y. Zhu, J. Lei, J. Wan, and L. Yu, "APNet: Adversarial learning assistance and perceived importance fusion network for all-day RGB-T salient object detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 4, pp. 957–968, Aug. 2022.
- [7] R. Cong, W. Song, J. Lei, G. Yue, Y. Zhao, and S. Kwong, "PSNet: Parallel symmetric network for video salient object detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 2, pp. 402–414, Apr. 2023.
- [8] P. Cascarano, L. Calatroni, and E. L. Piccolomini, "Efficient ℓ^0 gradient-based super-resolution for simplified image segmentation," *IEEE Trans. Comput. Imag.*, vol. 7, pp. 399–408, 2021.
- [9] Y. Xu, H. Nagahara, A. Shimada, and R.-I. Taniguchi, "Transcut2: Transparent object segmentation from a light-field image," *IEEE Trans. Comput. Imag.*, vol. 5, no. 3, pp. 465–477, Sep. 2019.
- [10] P. Zhuang, H. Li, S. Tan, B. Li, and J. Huang, "Image tampering localization using a dense fully convolutional network," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2986–2999, 2021.
- [11] S. Li, S. Xu, W. Ma, and Q. Zong, "Image manipulation localization using attentional cross-domain CNN features," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5614–5628, Sep. 2023.
- [12] H. Chen, C. Chang, Z. Shi, and Y. Lyu, "Hybrid features and semantic reinforcement network for image forgery detection," *Multimedia Syst.*, vol. 28, no. 2, pp. 363–374, 2022.
- [13] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [14] X. Lin et al., "Image manipulation detection by multiple tampering traces and edge artifact enhancement," *Pattern Recognit.*, vol. 133, 2023, Art. no. 109026.
- [15] F. Li, Z. Pei, X. Zhang, and C. Qin, "Image manipulation localization using multi-scale feature fusion and adaptive edge supervision," *IEEE Trans. Multimedia*, vol. 25, pp. 7851–7866, 2023.
- [16] P. Zhou, X. Han, V. I. Morariu, and L. S. Davis, "Learning rich features for image manipulation detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1053–1061.
- [17] B. Bayar and M. C. Stamm, "Constrained convolutional neural networks: A new approach towards general purpose image manipulation detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 11, pp. 2691–2706, Nov. 2018.
- [18] L. Zhuo, S. Tan, B. Li, and J. Huang, "Self-adversarial training incorporating forgery attention for image forgery localization," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 819–834, 2022.
- [19] C. Dong, X. Chen, R. Hu, J. Cao, and X. Li, "MVSS-Net: Multi-view multi-scale supervised networks for image manipulation detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3539–3553, Mar. 2023.
- [20] D. Xu, X. Shen, and Y. Lyu, "Up-net: Uncertainty-supervised parallel network for image manipulation localization," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 11, pp. 6390–6403, Nov. 2023.
- [21] D. Xu, X. Shen, Z. Shi, and N. Ta, "Semantic-agnostic progressive subtractive network for image manipulation detection and localization," *Neurocomputing*, vol. 543, 2023, Art. no. 126263.
- [22] Y. Zeng, B. Zhao, S. Qiu, T. Dai, and S.-T. Xia, "Towards effective image manipulation detection with proposal contrastive learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 9, pp. 4703–4714, Sep. 2023.
- [23] J. Zhou, X. Ma, X. Du, A. Y. Alhammedi, and W. Feng, "Pre-training-free image manipulation localization through non-mutually exclusive contrastive learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 22346–22356.
- [24] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 6000–6010.

- [25] Y. Wu, W. AbdAlmageed, and P. Natarajan, "Mantra-net: Manipulation tracing network for detection and localization of image forgeries with anomalous features," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9543–9552.
- [26] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "Segformer: Simple and efficient design for semantic segmentation with transformers," in *Proc. Neural Inf. Process. Syst.*, 2021, pp. 12077–12090.
- [27] Z. Li, J. Yang, B. Wang, Y. Li, and T. Pan, "Maskformer with improved encoder-decoder module for semantic segmentation of fine-resolution remote sensing images," in *Proc. 2022 IEEE Int. Conf. Image Process.*, 2022, pp. 1971–1975.
- [28] B. Chen, Y. Liu, Z. Zhang, G. Lu, and A. W. K. Kong, "Transattunet: Multi-level attention-guided u-net with transformer for medical image segmentation," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 55–68, Feb. 2024.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [30] D. Shullani, M. Fontani, M. Iuliani, O. A. Shaya, and A. Piva, "Vision: A video and image dataset for source identification," *EURASIP J. Inf. Secur.*, vol. 2017, no. 1, pp. 1–16, 2017.
- [31] Camera model identification, (n.d.). [Online]. Available: <https://www.kaggle.com/c/sp-society-camera-model-identification>
- [32] J. Dong, W. Wang, and T. Tan, "Casia image tampering detection evaluation database," in *Proc. 2013 IEEE China Summit Int. Conf. Signal Inf. Process.*, 2013, pp. 422–426.
- [33] B. Wen, Y. Zhu, R. Subramanian, T.-T. Ng, X. Shen, and S. Winkler, "Coverage—A novel database for copy-move forgery detection," in *Proc. 2016 IEEE Int. Conf. Image Process.*, 2016, pp. 161–165.
- [34] Nist: Nist nimble, Datasets. 2016. [Online]. Available: <https://www.nist.gov/itl/iad/mig/>
- [35] Y.-F. Hsu and S.-F. Chang, "Detecting image splicing using geometry invariants and camera characteristics consistency," in *Proc. Int. Conf. Multimedia and Expo*, 2006.
- [36] A. Novozámský, B. Mahdian, and S. Saic, "IMD2020: A large-scale annotated dataset tailored for detecting manipulated images," in *Proc. 2020 IEEE Winter Appl. Comput. Vis. Workshops*, 2020, pp. 71–80.
- [37] G. Mahfoudi, B. Tajini, F. Retraint, F. Morain-Nicolier, J. L. Dugelay, and M. Pic, "Defacto: Image and face manipulation dataset," in *Proc. 27th Eur. Signal Process. Conf.*, 2019, pp. 1–5.
- [38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [39] X. Hu, Z. Zhang, Z. Jiang, S. Chaudhuri, Z. Yang, and R. Nevatia, "Span: Spatial pyramid attention network for image manipulation localization," in *Proc. Comput. Vis.—ECCV 16th Eur. Conf.*, Aug. 2020, pp. 312–328.



Yan Xiang received the B.E. degree in electronic information engineering from Xiangtan University, Xiangtan, China, and the M.E. degree in circuits and systems from South China Normal University, Guangzhou, China. She is currently working toward the Ph.D. degree with the School of Computer Science and Engineering, Faculty of Innovation Engineering, Macau University of Science and Technology, Macau SAR, China. Her research interests include digital image forensics, computer vision, and deep learning techniques and applications.



Xiaochen Yuan (Senior Member, IEEE) received the Ph.D. degree in software engineering from the University of Macau, Macao, China, in 2013. From 2014 to 2015, she was a Postdoctoral Fellow with the Department of Computer and Information Science, University of Macau. From 2016 to 2021, she was an Assistant Professor and Associate Professor with the Faculty of Information Technology, Macau University of Science and Technology, Macao. She is currently an Associate Professor with the Faculty of Applied Sciences, Macao Polytechnic University, Macao. Her research interests include multimedia forensics and security, digital watermarking, AI model security, quantum watermarking, remote image processing, and deep learning techniques and applications.



Kaiqi Zhao received the Ph.D. degree in computer technology and application from the Macau University of Science and Technology, Macau, China, in 2024. She is currently with the School of Cyberspace Security, Shandong University of Political Science and Law, Jinan, China. Her research interests include image processing, digital forensics, and deep learning.



Tong Liu (Graduate Student Member, IEEE) received the B.Sc. degree in communication engineering from the Communication University of Zhejiang, Hangzhou, China, in 2018, and the M.S. degree in computer information science from the Macau University of Science and Technology, Macau, China, in 2020. She is currently working toward the Ph.D. degree in computer applied technology with the Faculty of Applied Sciences, Macau Polytechnic University, Macau. From 2021 to 2022, she was a Research Assistant with the Department of Computer and Information Science, University of Macau, Macau. Her research interests include digital image forensics, digital watermarking, tampering detection and self-recovery, and deep learning techniques and applications.



Zhiyao Xie received the B.S. degree from Beijing Normal University, Zhuhai, China, and the M.E. degree from the Macau University of Science and Technology, Macau, China. She is currently working toward the Ph.D. degree in computer applied technology with the Faculty of Applied Sciences, Macau Polytechnic University, Macau. Her research focuses on image tamper detection based on deep learning.



Guoheng Huang received the Ph.D. degree in software engineering from the University of Macau, Macao, China, in 2017. He is currently a CCF Member and Associate Professor of computer science with the Guangdong University of Technology, Guangzhou, China. His research interests include computer vision, pattern recognition, and artificial intelligence.



Jianqing Li (Senior Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 1999. From 2000 to 2002, he was a Visiting Professor with Information and Communications University, Daejeon, South Korea. From 2002 to 2004, he was a Research Fellow with Nanyang Technological University, Singapore. In August 2004, he joined the Macau University of Science and Technology, Macao, China. He is currently a Professor. His research interests include wireless networks, IoT, two-dimensional materials for photonics, and fiber sensors.